

Real-Time Shadow Mapping with Blinn-Phong Illumination

Claudio Pagnini

claudio.pagnini@stud.unifi.it

Abstract

This report documents a real-time rendering project built on OpenGL 3.3 Core Profile. The scene consists of three spheres of different sizes resting on a flat floor plane, illuminated by a directional light and rendered with real-time shadows. The implementation covers the full rasterization pipeline, coordinate, frame transformations, projection, rasterization, varying-variable interpolation, Blinn-Phong shading, texture mapping, and shadow mapping—tying each technical choice back to the theoretical concepts covered in the course lectures. A three-panel split-screen view compares shadow rendering without any correction, with an adaptive depth bias, and with Percentage Closer Filtering (PCF).

1 Introduction

The project is a real-time renderer written in C++17 using OpenGL 3.3 Core Profile, GLFW for windowing and input handling, GLM for linear-algebra types, and `stb_image` for texture loading. All shading logic is implemented in GLSL 3.30 vertex and fragment shaders.

The central technique is **shadow mapping**: producing accurate, plausible shadows at interactive frame rates using only the rasterization pipeline, without any form of ray tracing or global illumination. Beyond shadow casting, the renderer demonstrates perspective-correct texture mapping on a parametric sphere, a Blinn-Phong illumination model with material differentiation between the floor and the spheres, and an interactive first-person camera.

Rather than being an isolated exercise, the project deliberately touches every major stage of the pipeline taught in the course: coordinate frames, MVP transform, projection, clipping, back-face culling, rasterization, varying-variable interpolation, illumination models, texture mapping, and depth testing.

2 The OpenGL Rendering Pipeline

OpenGL exposes the GPU as a sequence of stages, some of which are programmable (vertex shader, fragment shader) and others fixed-function (primitive assembly, rasterization, depth test). The CPU prepares geometry and state, issues a draw call, and the GPU processes every vertex and every rasterized fragment independently and in parallel.

In this project the pipeline is executed **twice per frame**, which is the defining characteristic of shadow mapping [3]:

1. **Shadow pass.** The scene is rendered from the light's point of view into a depth-only framebuffer object (FBO). No colour is written; only the depth buffer is filled, producing the *shadow map*.
2. **Main render pass.** The scene is rendered from the camera's point of view. For each fragment the shader queries the shadow map to determine whether the point is lit or in shadow, then evaluates the full Blinn-Phong model.

Listing 1: Two-pass loop structure — `Shadow Mapping.cpp`

```
1 // --- PASS 1: shadow pass ---
2 glCullFace(GL_FRONT);
3 glViewport(0, 0, map.getShadowWidth(), map.getShadowHeight());
4 glBindFramebuffer(GL_FRAMEBUFFER, map.depthMapFBO);
5 glClear(GL_DEPTH_BUFFER_BIT);
6 shadowShader.use();
```

```

7 shadowShader.setMat4("lightSpaceMatrix", map.getLightSpaceMatrix());
8 // ... draw all scene objects ...
9 glBindFramebuffer(GL_FRAMEBUFFER, 0);
10 glCullFace(GL_BACK);
11
12 // --- PASS 2: main render pass ---
13 glViewport(0, 0, WIDTH, HEIGHT);
14 glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
15 shader.use();
16 // ... upload uniforms, draw all scene objects ...

```

The `glViewport` call uses the shadow map resolution (1024×1024) in the first pass and the screen resolution (1980×1080) in the second, because the viewport must always match the current render target.

3 Frames of Reference and the MVP Transform

Every geometric object lives in its own **object frame** $\tilde{o}$. The course defines coordinate frames using the relation

$$\tilde{o} = \tilde{w} O$$

where $\tilde{w}$ is the world frame and O is the matrix whose columns are the axes and origin of the object frame expressed in world coordinates. A point $\mathbf{p}_o$ in object coordinates maps to world coordinates as

$$\mathbf{p}_w = O^{-1} \mathbf{p}_o.$$

Analogously the eye (camera) frame $\tilde{e} = \tilde{w} E$ gives the view matrix $V = E$, so that

$$\mathbf{p}_e = E \mathbf{p}_w = E O^{-1} \mathbf{p}_o.$$

The combined model–view–projection transform is therefore

$$\mathbf{p}_{\text{clip}} = P \cdot V \cdot M \cdot \mathbf{p}_{\text{object}} = P \cdot E \cdot O^{-1} \cdot \mathbf{p}_{\text{object}},$$

where $M = O^{-1}$ is the model matrix and P is the projection matrix. In the vertex shader this translates directly to:

Listing 2: MVP transform — `basic.vert`

```

1 FragPos      = vec3(model * vec4(aPos, 1.0));           // object -> world
2 gl_Position  = projection * view * vec4(FragPos, 1.0); // world -> clip

```

3.1 Normal Transform

Normals cannot be transformed by M alone. After a non-uniform scale, the naive transform $M\mathbf{n}$ is no longer perpendicular to the surface. The correct transform is the inverse-transpose of the upper-left 3×3 sub-matrix of M , derived from the perpendicularity condition $\mathbf{n}' \cdot (M\mathbf{t}) = 0$ [4]:

$$\mathbf{n}' = (M^{-1})^\top \mathbf{n}.$$

Listing 3: Normal correction — `basic.vert`

```

1 Normal = mat3(transpose(inverse(model))) * aNormal;

```

4 Projection

4.1 Perspective Projection — Camera

The perspective projection matrix P maps eye-space coordinates to clip space by placing $w_c = -z_e$ in the fourth component of the output vector. The subsequent **perspective divide** ($\div w_c$), performed automatically by the hardware after the vertex shader, yields the Normalised Device Coordinates (NDC):

$$x_n = \frac{x_c}{w_c} = \frac{x_e}{-z_e}, \quad y_n = \frac{y_c}{w_c} = \frac{y_e}{-z_e}.$$

This is the projection predicted by the pinhole camera model. The full matrix, including the third row that maps z_e to $[-1, +1]$ for the depth buffer, is constructed by `glm::perspective`.

Listing 4: Perspective projection — Shadow Mapping.cpp

```

1 glm::mat4 projection = glm::perspective(
2   glm::radians(fov),    // vertical field of view (1 to 45 degrees)
3   aspectRatio,
4   0.1f, 100.0f          // near and far planes
5 );

```

4.2 Orthographic Projection — Light

A directional light source has parallel rays; using a perspective projection for the light would introduce foreshortening that distorts the shadow map. An orthographic projection preserves parallelism and is therefore the correct choice [3]:

Listing 5: Orthographic projection for the light — ShadowMap.cpp

```

1 // "We are modelling a directional light source and all its light rays are
2 // parallel. That's why we have to use an orthographic projection matrix."
3 glm::mat4 lightProjection = glm::ortho(
4   -10.0f, 10.0f, -10.0f, 10.0f,
5   near_plane, far_plane
6 );

```

5 Rasterization and Varying Variables

5.1 Back-Face Culling

Before rasterization, back-facing triangles are discarded. Vertices are stored in the VBO in counter-clockwise (CCW) order when viewed from outside the object. After the perspective divide, the GPU computes the z -component of the cross product of two edge vectors in NDC:

$$c_z = (x_n^3 - x_n^2)(y_n^1 - y_n^2) - (y_n^3 - y_n^2)(x_n^1 - x_n^2).$$

If $c_z < 0$ the triangle is back-facing and is culled. During the shadow pass, culling is inverted (`glCullFace(GL_FRONT)`) to reduce self-shadowing artefacts.

5.2 Perspective-Correct Interpolation

Once a triangle survives culling, the rasterizer computes barycentric coordinates ($\lambda_0, \lambda_1, \lambda_2$) for each covered pixel. Naively interpolating a varying v with barycentric weights in screen space produces incorrect results after a perspective projection, because the projection is not an affine map.

The course shows that, even though v is not affine in window coordinates, the quantities v/w_c and $1/w_c$ are [2]. This follows from the fact that v is affine in object space, the MVP is linear in homogeneous coordinates, and dividing through by w_c preserves this linearity in NDC:

$$\frac{v}{w_c} = ax_n + by_n + cz_n + d.$$

The rasterizer therefore interpolates v/w_c and $1/w_c$ linearly, then recovers v at each pixel:

$$v = \frac{v/w_c}{1/w_c}.$$

OpenGL performs this automatically for every out variable declared in the vertex shader.

Listing 6: Varying outputs — basic.vert

```

1 out vec3 FragPos;           // world-space position
2 out vec3 Normal;           // world-space normal
3 out vec4 FragPosLightSpace; // position in light clip space (shadow lookup)
4 out vec2 TexCoords;        // UV texture coordinates

```

5.3 GPU Vertex Layout

The `Mesh` class packs position, normal, and UV into a contiguous `Vertex` struct and registers the attribute pointers once in `setupMesh()`:

Listing 7: Attribute pointers — `Mesh.cpp`

```
1 // Position      attribute 0, 3 floats, offset 0
2 glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, sizeof(Vertex), (void*)0);
3 // Normal        attribute 1
4 glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, sizeof(Vertex),
5                       (void*)offsetof(Vertex, Normal));
6 // TexCoords     attribute 2
7 glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, sizeof(Vertex),
8                       (void*)offsetof(Vertex, TexCoords));
```

6 Blinn-Phong Illumination

The fragment shader evaluates a local illumination model composed of three terms.

Ambient. A constant low-intensity term preventing fully unlit surfaces from appearing completely black:

$$L_{\text{amb}} = k_a \cdot L_i.$$

Diffuse. Lambert’s cosine law models energy scattered equally from a matte surface:

$$L_{\text{diff}} = \max(\hat{N} \cdot \hat{L}, 0) \cdot L_i.$$

Specular (Blinn). Instead of reflecting $\hat{L}$ around $\hat{N}$ (Phong), the Blinn variant uses the *halfway vector* $\hat{h} = \text{norm}(\hat{L} + \hat{V})$:

$$L_{\text{spec}} = \max(\hat{N} \cdot \hat{h}, 0)^n \cdot L_i, \quad n = 720.$$

Listing 8: Blinn-Phong — `basic.frag`

```
1 vec3 N = normalize(Normal);
2 vec3 L = normalize(-lightDir);
3 vec3 V = normalize(viewPos - FragPos);
4
5 float diff = max(dot(N, L), 0.0); // Lambert
6
7 vec3 specular = vec3(0.0);
8 if (!isFloor) { // floor is matte
9     vec3 h = normalize(L + V);
10    float spec = pow(max(dot(N, h), 0.0), 720.0);
11    specular = spec * lightColor;
12 }
```

7 Texture Mapping

Texture mapping assigns a 2D image to a surface through UV coordinates $(u, v) \in [0, 1]^2$. Because UV is a vertex attribute, it is affine in object space and is correctly interpolated by the rasterizer via perspective-correct barycentric weights.

For the parametric sphere, UV coordinates are generated directly from the longitude/latitude angles:

Listing 9: UV generation — `Sphere.cpp`

```
1 float s = (float)j / sectorCount; // u in [0,1] (longitude)
2 float t = (float)i / stackCount;  // v in [0,1] (latitude)
3 vertex.TexCoords = glm::vec2(s, t);
```

The marble texture is applied to the rotating middle sphere. The lookup in the fragment shader is:

Listing 10: Texture lookup — `basic.frag`

```
1 if (useTexture)
2     objectColor = texture(diffuseTexture, TexCoords).rgb;
```

8 Shadow Mapping

Shadow mapping is conceptually an application of *projector mapping*: the shadow map texture is "projected" onto the scene from the light's viewpoint. A fragment is in shadow if its depth, as seen from the light, exceeds the value stored in the shadow map at the corresponding projected UV location [3].

8.1 Pass 1 — Depth Map Generation

The shadow map is a 1024×1024 floating-point depth texture attached to a dedicated FBO:

Listing 11: FBO and depth texture — ShadowMap.cpp

```
1 glGenFramebuffers(1, &depthMapFBO);
2 glGenTextures(1, &depthMap);
3 glBindTexture(GL_TEXTURE_2D, depthMap);
4 glTexImage2D(GL_TEXTURE_2D, 0, GL_DEPTH_COMPONENT,
5             SHADOW_WIDTH, SHADOW_HEIGHT, 0,
6             GL_DEPTH_COMPONENT, GL_FLOAT, NULL);
7 glFramebufferTexture2D(GL_FRAMEBUFFER, GL_DEPTH_ATTACHMENT,
8                       GL_TEXTURE_2D, depthMap, 0);
9 glDrawBuffer(GL_NONE); // no colour output needed
10 glReadBuffer(GL_NONE);
```

Front faces are culled during the shadow pass so that the stored depths correspond to back faces, which are slightly further from the light. This reduces *shadow acne* — the self-shadowing artefact visible in the middle panel of Figure 1.

8.2 Pass 2 — Shadow Lookup

Each fragment is transformed into light clip space via `FragPosLightSpace`, then the perspective divide and a remap to $[0, 1]$ yield the shadow-map UV:

Listing 12: Projector coordinate recovery — basic.frag

```
1 vec3 projCoords = FragPosLightSpace.xyz / FragPosLightSpace.w; // / w_t
2 projCoords = projCoords * 0.5 + 0.5; // NDC [-1,1] -> UV [0,1]
```

8.3 Artefact Correction: Bias and PCF

Two common artefacts arise in basic shadow mapping:

- **Shadow acne**: a surface self-shadows due to floating-point precision. An adaptive bias whose magnitude grows with the angle of incidence eliminates it [1]:

$$\text{bias} = \max(0.01 (1 - \hat{N} \cdot \hat{L}), 0.005).$$

- **Hard edges**: the binary shadow test produces aliased shadow boundaries. **Percentage Closer Filtering (PCF)** [5] averages the shadow test over a 3×3 neighbourhood of shadow-map texels, producing soft shadow boundaries:

Listing 13: PCF — basic.frag

```
1 float bias = max(0.01 * (1.0 - dot(N, L)), 0.005);
2 float shadow = 0.0;
3 vec2 texelSize = 1.0 / textureSize(shadowMap, 0);
4 for (int x = -1; x <= 1; ++x)
5     for (int y = -1; y <= 1; ++y) {
6         float pcfDepth = texture(shadowMap,
7                                 projCoords.xy + vec2(x,y)*texelSize).r;
8         shadow += (currentDepth - bias > pcfDepth) ? 1.0 : 0.0;
9     }
10 shadow /= 9.0;
```

PCF can be understood as a box filter applied to the binary visibility function, analogous to the sampling and filtering concepts introduced in Lecture 8B.

The three rendering modes are shown side by side in the split-screen output (Figure 1).

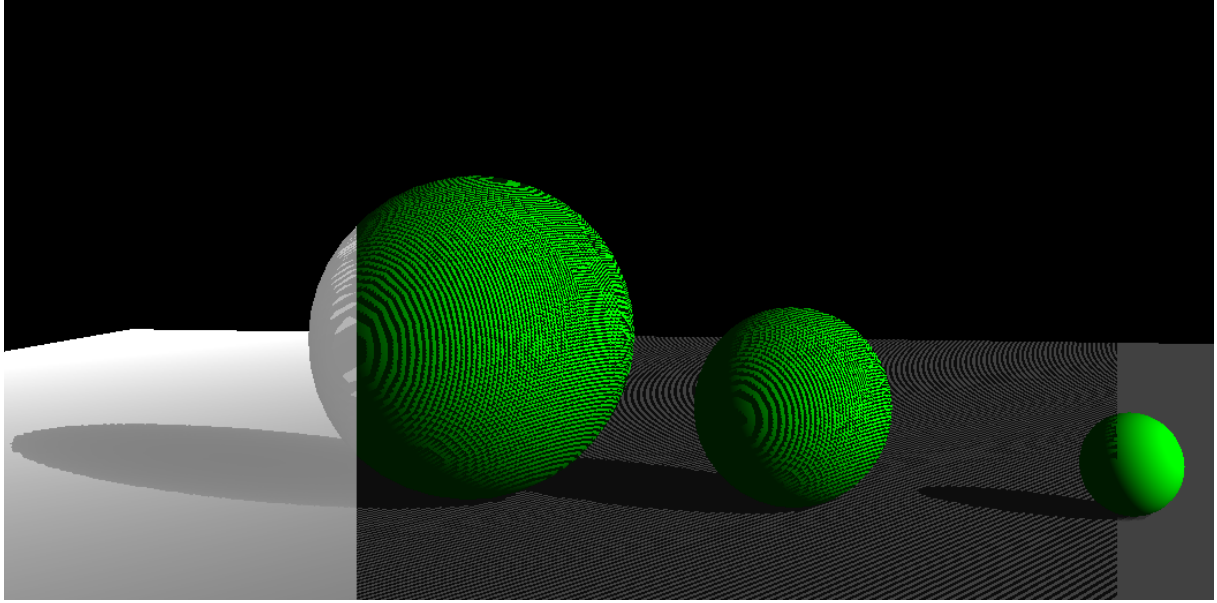


Figure 1: Left: depth map visualisation. Centre: hard shadows without bias (shadow acne visible). Right: soft shadows with adaptive bias and PCF.

9 Conclusions

This project implements a real-time renderer that exercises the full rasterization pipeline, from coordinate-frame transformations through to per-fragment shading and depth testing. The two-pass shadow-mapping technique ties together projection, varying-variable interpolation, and depth testing in a single working application.

The three-panel split-screen view provides a self-contained illustration of common shadow-mapping artefacts and their corrections. Natural extensions would include omnidirectional shadow maps (cube-map depth texture) for point lights, physically-based rendering materials, or deferred shading to support many light sources without a per-light render pass.

References

- [1] de Vries, J. *LearnOpenGL — Shadow Mapping*. <https://learnopengl.com/Advanced-Lighting/Shadows/Shadow-Mapping> (2014–2024).
- [2] Course slides: *Lect 07-A — Varying Variables*; *Lect 06-A — From Vertex to Pixel: Clipping & Culling*; *Lect 06-B — Viewport & Rasterization*. University of Florence, Computer Graphics, A.Y. 2025/2026.
- [3] Course slides: *Lect 08-A — Texture Mapping (shadow mapping and projector mapping sections)*. University of Florence, Computer Graphics, A.Y. 2025/2026.
- [4] Shirley, P.; Marschner, S. *Fundamentals of Computer Graphics*, 4th ed. CRC Press, 2015.
- [5] Reeves, W. T.; Salesin, D. H.; Cook, R. L. Rendering antialiased shadows with depth maps. *SIG-GRAPH Computer Graphics*, 21(4):283–291, 1987.